

这是一个非常核心的架构问题。简单来说:**SGLang** 不是只能私有化部署, 它可以无缝对接外部的商业公共 **API**(如 **OpenAI**); 而 **vLLM** 则是纯粹的私有化部署(自托管)推理引擎。
我们来详细拆解这两者的区别:

1. **SGLang**: 既能“私有化部署”, 也能“白嫖/优化商业 **API**”

SGLang 的设计初衷是一个**“大模型程序编程框架 + 运行时”**, 它天然分为前端和后端, 这赋予了它极大的灵活性:

- 路线 **A**: 私有化部署(自托管模型)
就像你在课程资料中看到的, 你可以下载开源模型(如 Qwen、Llama), 使用 SGLang 的后端(依赖 GPU 和 CUDA)在本地或企业内部服务器上拉起一个推理服务。它通过 RadixAttention 等技术让本地模型跑得飞快。
- 路线 **B**: 调用公共/闭源商业 **API**(如 **GPT-4, Claude**)
SGLang 的前端语言(Python 接口)完全可以脱离本地 GPU, 直接将后端的执行目标(Backend)指向 OpenAI、Anthropic 等公共 API。
 - 核心杀手锏(**API**推测执行): 正如课程大纲中提到的, SGLang 针对商业 API 有专门的优化。当你使用 SGLang 写了复杂的 Agent 逻辑去调用 GPT-4 时, SGLang 可以在本地进行**“API 推测执行”。它会预测后续可能需要的 **Token** 结构, 合并多次 **API** 请求, 从而大幅减少调用商业 API 的次数和成本**, 同时降低网络延迟。

2. **vLLM**: 纯粹的“私有化部署”推理引擎

vLLM 的定位非常明确, 它就是一台纯粹的“发动机”。

- 只能自托管: 使用 vLLM 的前提是, 你必须拥有模型的权重文件(**Weights**)并且拥有承载这些权重的算力(**GPU**)。它的核心技术(PagedAttention、Continuous Batching)全是直接操作 GPU 显存和物理硬件的。
- 不能代理外部 **API**: 你不能用 vLLM 去“配置”或“调用”OpenAI 的 GPT-4。vLLM 不具备这样的路由代理功能。
- 提供兼容 **API**: vLLM 的作用是把部署在本地的开源模型, 包装成一个对外提供服务的 API 接口(通常是 OpenAI 兼容格式)。也就是说, 它让你自己变成了一个“小型的 OpenAI”, 供别人或你的前端应用来调用。

总结对比

特性	vLLM	SGLang
核心定位	极致的高吞吐推理引擎(让本地模型跑得快)	复杂 LLM 任务运行时(让业务逻辑跑得快、成本低)
是否支持私有化部署	✔ 是(必须自己有 GPU 和开源模型)	✔ 是(使用 SGLang-Server 高效部署开源模型)
是否支持调用商业 API	✘ 否(不支持调用外部闭源模型)	✔ 是(可指向 GPT-4 等, 并有 API 成本与速度优化功能)

通俗的比喻:

- **vLLM** 就像是一家工厂底层的超级流水线机器。你必须自己买厂房(GPU)、买原材料(开源模型权重), 然后这台机器能帮你最高效地生产出产品(Token)。

- **SGLang** 就像是一个高级包工头(前端)+ 自己的生产车间(后端)。他既可以指挥自己车间里的流水线干活(私有化部署), 如果遇到自己干不了的精细活儿, 他也能极其聪明、省钱地把任务外包给隔壁的顶级代工厂(调用外部商业 API)。